

Entwicklerhandbuch für die Umsetzung des Brettspieles 'Go'

Harry Flohr (6811837)
und
Jan Ole Wellnitz (6796226)

19. September 2017

Inhaltsverzeichnis

1	Strukturen	3
1.1	Spielfeld	3
1.2	World (Spielzustand)	4
1.3	Spielzustände	4
2	Module der Software	5
2.1	Bildverarbeitung (go_bv.rkt)	5
2.2	Spielbrett zeichnen (go_draw_board.rkt)	6
2.3	Spielclient (go_client.rkt)	9
2.4	Spielserver (go_server.rkt)	9
2.5	Spiellogik (go_logic.rkt)	10
3	Schnittstellen	12
3.1	Bildverarbeitung	12
3.2	Spielbrett zeichnen	12
3.3	Spielclient	13
3.4	Spielserver	13
3.5	Spiellogik	14
4	Paketstruktur	15
5	Programmabläufe	16
5.1	Programmablauf	16
5.2	Stein setzen	17
5.3	Auswertung	19
5.4	Gebietszuweisung	21
6	Beispiele	23
6.1	Erlaubter Selbstmord	23
6.2	Auswertung	24
7	ToDo's	25
7.1	Dateiverarbeitung (go_datei.rkt)	25
7.2	Bug in Auswertung	25

1 Strukturen

1.1 Spielfeld

Ein Spielfeld besteht aus 19x19 Felder, auf denen Steine positioniert werden können. Das Spielfeld wird in dem Programm als eine Liste mit 19 weiteren Listen dargestellt. Diese 19 Listen stehen für die y-Koordinaten des Spielfeldes. Jede dieser 19 Listen enthält 19 Positionen, um einen Wert einzutragen. Diese Unterlisten stehen für die x-Koordinaten.

Wenn an einer Position eine 0 steht, befindet sich kein Stein auf dem Spielfeld. Befindet sich dort eine 1, liegt an der Position ein weißer Stein. Bei einer -1, liegt an der Position ein schwarzer Stein.

Darstellung des Spielstandes in Abbildung 2.3:

```
'((0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 1 0 0 -1 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 1 -1 0 0 -1 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 1 -1 0 1 -1 -1 0 0 0 0 0 0 0)
(0 0 0 0 0 1 1 -1 0 1 1 -1 -1 1 0 0 0 0 0)
(0 0 0 0 0 1 -1 -1 0 1 -1 0 0 0 0 0 0 0 0)
(0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 -1 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 -1 -1 -1 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 -1 1 1 -1 0 0 0 0 0 0)
(0 0 0 0 1 1 1 1 1 -1 1 1 -1 0 1 1 0 0 0)
(0 0 0 0 1 -1 -1 -1 -1 1 0 1 0 -1 0 -1 1 0 0)
(0 0 0 1 -1 0 -1 1 1 0 0 1 -1 -1 -1 1 0 0 0)
(0 0 0 -1 0 0 -1 1 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 -1 1 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 -1 1 0 0 0 0 0 0 0 0 0 0 0))
```

1.2 World (Spielzustand)

Die World setzt sich aus mehreren Informationen zu einer Liste zusammen:

1. Spielfeld (siehe 1.1)
2. Weltzustand
Der Weltzustand gibt an, ob der Client ziehen darf, warten muss oder das Spiel beendet wurde (Zustände: wait, play, lost, won, started, setkilledblack, setkilledwhite, choosedraw, choosepasstatus, choosehandicap, usehandicap).
3. Geschlagene Steine
Die bereits geschlagenen Steine werden als Pair '(schwarz . weiß) übergeben.
4. Gepasster Spielzug
Hat der vorherige Spieler in seinem Spielzug gepasst und der Spieler an der Reihe passt ebenfalls, ist das Spiel beendet und die Auswertung startet (Gepasst: true/false).

Als Beispiel eine mögliche World-Darstellung bei der der aktuelle Spieler am Zug ist, 5 schwarze und 8 weiße Steine geschlagen wurden und der vorherige Spieler nicht gepasst hat. Der Spielstand ist zur Übersicht vereinfacht:

```
'(play '(spielfeld) '(5 . 8) false)
```

1.3 Weltzustände

1. wait: Abwarten
2. play: An der Reihe
3. won: Gewonnen
4. lost: verloren
5. started: Wahl des Spielstarts als neues Spiel oder aus einer Bilddatei
6. setkilledblack: Eingabe der geschlagenen schwarzen Steine bei Spielstart
7. setkilledwhite: Eingabe der geschlagenen weißen Steine bei Spielstart
8. choosedraw: Wahl der Zugreihenfolge
9. choosepasstatus: Wahl ob vor Spielstart gepasst wurde
10. choosehandicap: Setzen der Höhe der Vorgabe
11. usehandicap: Setzen der Vorgabe auf dem Spielfeld

2 Module der Software

2.1 Bildverarbeitung (go_bv.rkt)

Das Modul Bildverarbeitung ist für das Auslesen der Spielfeldbelegung eines Bildes zuständig. Ein Bild wird in das Modul geladen und dann verarbeitet. Es wird zuerst das Spielbrett aus dem Bild extrahiert und in ein Schwarz-Weiß-Bild gewandelt. Die Graustufen stellen die Anteile der blauen Farbe in dem originalen Bild dar. Das Bild wird anschließend noch verwaschen, um die Spielfeldlinien und Reflexionen leichter von den Spielsteinen unterscheiden zu können.

In diesem Bild wird die Koordinaten des ersten Spielfeldes (oben links) gesucht. An dieser Koordinate wird geschaut, ob die Schwellenwerte für einen weißen oder schwarzen Stein getroffen wird. Diese Überprüfung wird an insgesamt neun Stellen pro Koordinate vorgenommen. Es wird quadratisch um die Koordinate kontrolliert (siehe Abb. 2.1).



Abbildung 2.1: Suchemuster an Koordinate

Dieser Prozess wird an jeder möglichen Koordinate für einen Stein durchgeführt und so eine Spielfeldbelegung, wie in 1.1 beschreiben, erstellt. Die Koordinaten werden anhand zweier Schrittweiten (in x - und y-Richtung) ermittelt. Ist das ganze Spielfeld kontrolliert, hat das Modul die vollständige Spielfeldbelegung des Bildes projiziert.

2.2 Spielbrett zeichnen (go_draw_board.rkt)

Das Modul `go_draw_board.rkt` ist für die Anzeige des gesamten User-Interfaces zuständig.

Es zeichnet zum einen aus einer gültigen Spielfeldbelegung (1.1) das belegte Spielbrett. Zuerst wird das Grundbrett gezeichnet. Anschließend werden die Listen des Spielstandes ausgelesen und die benötigten Steine auf das Spielfeld gezeichnet. Die Leserichtung der Listen erfolgt von oben links nach unten rechts.

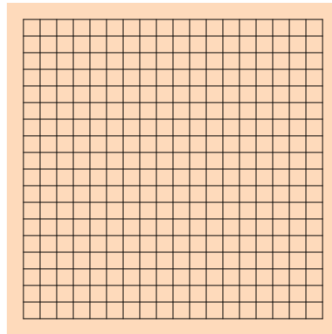


Abbildung 2.2: Leeres Spielfeld

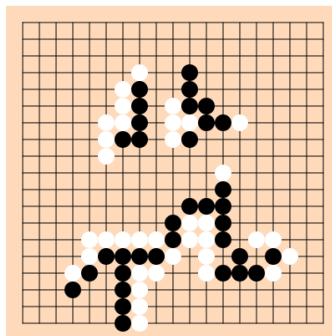
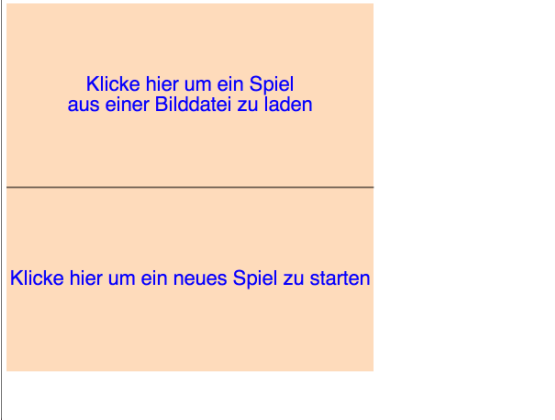


Abbildung 2.3: Belegtes Spielfeld

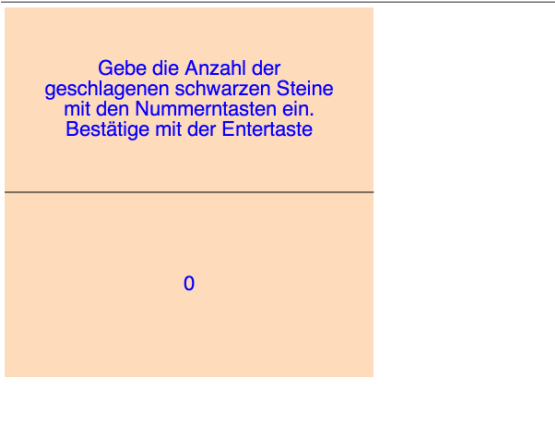
Zudem zeichnet das Modul beim Start eines Spieles die benötigten Auswahloptionen, die der Benutzer vor dem Starten eingeben muss. Das kann zum einen über das Darstellen einer Auswahlfläche (Abb. 2.4) geschehen, auf die der Benutzer klicken muss. Zum anderen aber auch eine Aufforderung zur Eingabe von Zahlen über die Tastatur (Abb. 2.5).



Klicke hier um ein Spiel
aus einer Bilddatei zu laden

Klicke hier um ein neues Spiel zu starten

Abbildung 2.4: Auswahl über Optionsfelder



Gebe die Anzahl der
geschlagenen schwarzen Steine
mit den Nummertasten ein.
Bestätige mit der Enter-taste

0

Abbildung 2.5: Zahleneingabe

Nach Eingabe aller Parameter erstellt die Komponente das User-Interface für das Spielen selbst. Dazu gehören das Spielfeld, Spielstatus des Spielers, die erzielten Punkte und Buttons (für Passen oder ein neues Spiel starten) (Abb. 2.6).

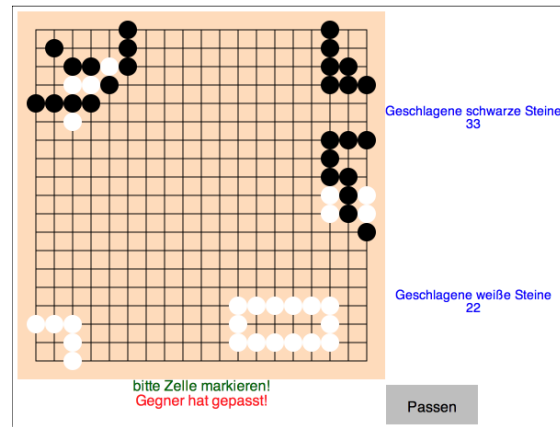


Abbildung 2.6: Das Spielfeld während des Spiels

Alle Darstellung des User-Interfaces können über die Schnittstellen des Moduls angezeigt werden. Soll eine z. B. Eingabeoption erstellt werden, muss dazu eine neue Funktion erstellt werden und als Schnittstelle bereit gestellt werden. Um bestehende Darstellungen zu ändern, müssen die entsprechenden Funktionen in diesem Modul angepasst werden.

2.3 Spielclient (go_client.rkt)

Der Spielclient steuert die Kommunikation des Spielers mit dem Server. Mit dem Spielclient kann auf zwei verschiedene Arten ein Spiel gestartet werden. Es kann ein komplett neues Spiel gestartet werden und es kann ein Spielstand mit einem Bild-Spielstand gestartet werden. Bei dem Start aus einem Bild-Spielstand fragt der Client ab, wie viele Steine bereits geschlagen wurden, welche Farbe der Spieler hat, ob ein Spieler gepasst hat und welcher Spieler am Zug ist. Diese Informationen werden dann zu dem Server geschickt und dieser initialisiert den ersten Spielstand (1.2). Die Funktionen zum zeichnen der Auswahloptionen und Eingaben stammen aus dem `go_draw_board.rkt` Modul.

Ebenfalls ist der Spielclient für die Erstellung des Interfaces zuständig. Über dieses zeigt er den Spielern während eines Spiels den aktuellen Spielstand an, ob sie an der Reihe sind, die Anzahl der geschlagenen Steine, ob der Gegner gepasst hat, sie einen ungültigen Zug vorgenommen haben und wer gewonnen hat. Auch die Buttons zum Passen oder für das Starten eines neuen Spiels, werden angezeigt. Diese Anzeige wird mit Hilfe des `go_draw_board.rkt` Modul erstellt.

Die Hauptaufgabe des Clients ist aber die Entgegennahme von Benutzereingaben. Der Client nimmt Click-Events und Tastatureingaben entgegen, leitet sie zum Server weiter, empfängt das Ergebnis und stellt das Ergebnis dar. So kann der Benutzer über den Client ein Spiel starten, die Spiel-Parameter eingeben, das Spiel bis zum Ende durchführen und anschließend ein neues Spiel starten.

2.4 Spielserver (go_server.rkt)

Der Spielserver leitet das Spiel. Zu Beginn erhält der Server die Information, ob er ein neues Spiel starten oder ein Spiel aus einem Bild laden soll. Der Server lädt das Bild und ermittelt den Spielstand. Der Server schickt dann ggf. eine Abfrage für die weiteren benötigten Informationen an den Client. Diese sind bei einem neuen Spiel die Vorgabe und bei einem Spielstand aus einem Bild die geschlagene Steine, der Spieler, der am Zug ist, und ob der vorherige Spieler gepasst hat. Anschließend sendet der Server den vollständigen Startzustand an die Clients.

Eine weitere Aufgabe während des gesamten Spieles ist die Verwaltung des Spielstandes (1.2) und welcher Spieler am Zug ist. Ebenfalls prüft der Server die übermittelten Spielzüge der Spieler auf ihre Gültigkeit und teilt den Spielern ggf. mit, dass der Spielzug nicht zulässig ist. Bei zulässigen Zügen passt der Server den Spielstand (1.2) an und übermittelt den vollständig erstellten Spielstand an die Clients.

Nach Beendigung eines Spieles, ermittelt der Server den Gewinner und teilt den Spielern (Clients) den Gewinner mit.

2.5 Spiellogik (go_logic.rkt)

In dem Modul `go_logic.rkt` ist die Spiellogik implementiert. Diese kann in drei verschiedenen Aufgaben unterteilt werden.

1. Vorgaben machen:

Wenn bei dem Start eines neuen Spiels eine Vorgabe gewährt wird, kann der Spieler vor Spielbeginn die angegebene Anzahl Steine setzen.

2. Stein setzen:

Wenn ein Stein gesetzt wird, muss geprüft werden, dass dieser keinen Selbstmord begeht (Abb. 1.7). Das würde bedeuten, dass dieser Stein und ggf. seine anliegende Struktur durch das Setzen dieses Steines zerstört wird.

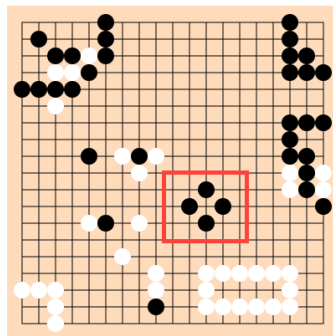


Abbildung 2.7: Selbstmord - Weißer Stein darf nicht gesetzt werden

Dieser Zug ist nur erlaubt, wenn mit diesem Zug Steine des Gegners geschlagen werden und so der Stein und ggf. seine Struktur neue Freiheiten erhalten.

Wenn ein Stein normal gesetzt wird, kann das Setzen zur Folge haben, dass durch diesen Zug eine Struktur des Gegners zerstört wird. Wenn dieser Fall eintritt müssen alle geschlagenen Steine von dem Spielfeld entfernt werden und die Punkte werden dem Spieler gut geschrieben.

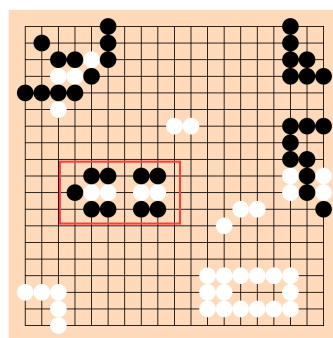


Abbildung 2.8: Spielbrett vor dem Schlagen der Steine

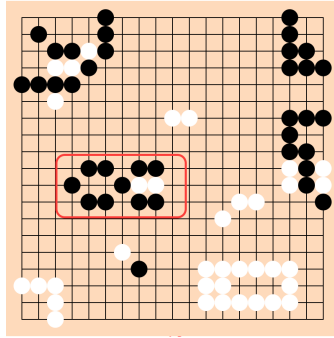


Abbildung 2.9: Spielbrett nach dem Schlagen der Steine

3. Auswertung:

Nachdem beide Spieler nacheinander gepasst haben, ist das Spiel beendet. Dann startet die Auswertung. Bei der Auswertung müssen eroberte Gebiete gefunden werden. Das sind Flächen auf denen kein Stein liegt, die aber von einem Spieler eingeschlossen sind. Wenn ein Spieler eine freie Fläche erobert hat, erhält dieser die Anzahl an freien Positionen sowie die Anzahl der gegnerischen Steine in diesem Gebiet. Ein erobertes Gebiet ist in Abb. zu sehen. Nachdem alle Flächen überprüft sind, wird die Anzahl der erzielten Punkte auf die bereits erzielten Punkte aufaddiert.

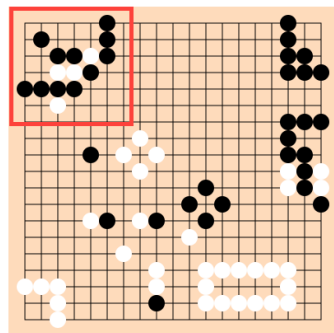


Abbildung 2.10: Gebiet von Schwarz eingenommen - 13 freie Flächen + 3 geschlagene weiße Steine

Diese drei Hauptaufgaben können über Schnittstellen ausgeführt werden. Änderungen in der Spiellogik bzw. Erweiterungen sind demzufolge in diesem Modul zu erledigen.

3 Schnittstellen

3.1 Bildverarbeitung

1. (board-state):
Das Modul bietet diese Funktion, um die Belegung eines Spielfeldes aus einem Bild heraus auszulesen. Der Dateipfad des Bildes ist in dem Modul selbst hinterlegt. Durch den Aufruf liefert die Funktion die Belegung des Spielfeldes, wie in 1.1 beschrieben, zurück.

3.2 Spielbrett zeichnen

1. (draw-board-with-score world):
Die Funktion zeichnet das User-Interface während des Spieles. Dazu gehören das Spielfeld mit der aktuellen Belegung, Anzahl der geschlagenen Steine, welcher Spieler am Zug ist, Passstatus des Gegners sowie ein Button zum Passen oder nach der Auswertung für den Start eines neuen Spieles. Auf dem Spielfeld kann anschließend auf die gewünschte Stelle für einen Stein ausgewählt werden, der Client verarbeitet diese Eingabe und schickt sie zu dem Server.
2. (start-field):
Diese Funktion zeichnet das User-Interface für die Auswahl, ob eine Datei aus der Bilddatei gestartet werden soll oder ob ein komplett neues Spiel gestartet wird. Dazu werden zwei Auswahlbereiche gezeichnet, die der Benutzer per Mausklick auswählen kann.
3. (choose-color-field):
Diese Funktion zeichnet das User-Interface für die Auswahl, mit welcher der auswählende Spieler spielt. Dazu werden zwei Auswahlbereiche gezeichnet, die der Benutzer per Mausklick auswählen kann.
4. (choose-killed-black-stones killed-stones):
Diese Funktion zeichnet das User-Interface für die Eingabe, wie viele schwarze Steine bereits geschlagen worden sind. Dazu wird ein Infotext sowie eine Eingabeanzeige gezeichnet. Die Eingabe erfolgt per Tastatur. Nach jeder Eingabe wird die Anzeige aktualisiert.
5. (choose-killed-white-stones killed-stones):
Diese Funktion zeichnet das User-Interface für die Eingabe, wie viele weiße Steine bereits geschlagen worden sind. Dazu wird ein Infotext sowie eine Eingabeanzeige

gezeichnet. Die Eingabe erfolgt per Tastatur. Nach jeder Eingabe wird die Anzeige aktualisiert.

6. (draw-final-score world):
Diese Funktion zeigt den Gewinner sowie erreichte Punktzahl beider Spieler an.
7. (choose-handicap handicap):
Diese Funktion zeichnet das User-Interface für die Eingabe, wie viele Steine ein Spieler als Vorlage legen darf. Dazu wird ein Infotext sowie eine Eingabeanzeige gezeichnet. Die Eingabe erfolgt per Tastatur. Nach jeder Eingabe wird die Anzeige aktualisiert.
8. (set-handicap world):
Diese Funktion ermöglicht das Setzen der Vorgabe. Dazu werden das Spielfeld und die Anzahl der Vorgabe angezeigt. Nach jedem gesetzten Stein wird die Anzeige aktualisiert.
9. (choose-draworder-field):
Diese Funktion zeichnet das User-Interface für die Auswahl, welcher Spieler zuletzt am Zug war. Dazu werden zwei Auswahlbereiche gezeichnet, die der Benutzer per Mausklick auswählen kann.
10. (choose-passstatus):
Diese Funktion zeichnet das User-Interface für die Auswahl, ob ein Spieler im letzten Zug vor der Fotoaufnahme gepasst hat. Dazu werden zwei Auswahlbereiche gezeichnet, die der Benutzer per Mausklick auswählen kann.

3.3 Spielclient

1. (receive w m):
Der Spielclient erhält über die von dem Universe-Paket gelieferten Funktion seinen Spielstand (1.2). In dem Parameter w befindet sich der alte Weltzustand und in dem Parameter m der neue Weltzustand. Der neue Weltzustand m wird zuerst überprüft. Ist er gültig wird dieser angezeigt, ist er es nicht, wird der alte Weltzustand gewählt.
2. (make-package w m):
Der Spielclient kann mittels der Funktion (make-package w m) mit dem Server kommunizieren. So kann der Spielclient mit dem Parameter w seine aktuelle Welt senden und in dem Parameter m weitere Informationen verpacken. Zum Beispiel wird dort angegeben an welcher Position der Spieler seinen Stein gesetzt hat.

3.4 Spielserver

1. (make-mail to content):
Der Spielserver kann mit der Funktion make-mail Informationen an den Client

senden. In dem Parameter `to` wird das Ziel des Aufrufes festgelegt. In dem `content` befindet sich der aktuelle Spielstand (1.2).

2. (`handle-messages univ wrld m`):

Der Server kann mit der Funktion `handle-messages` Daten von den Clients empfangen. In dem Parameter `univ` befindet sich das Universum selbst. In `wrld` befindet sich die von dem Client gesendete World und in dem Parameter `m` die Aktion, die der Client gerne ausführen möchte. Das kann z. B. das Setzen eines Steines sein. Dann befindet sich in `m` die Funktion `'set` sowie die `x`- und `y`-Koordinaten der Position, an der der Stein gesetzt werden soll. Ebenfalls können sich in dem Parameter `m` die Informationen für den Spielstart befinden. Wie z. B. die Vorgabe oder die geschlagenen Steine.

3. (`add-world univ wrld`):

Mit der Funktion `add-world` kann sich ein Client bei dem Server anmelden. Dazu wird das Universum (`univ`) sowie die World (`wrld`) benötigt. Wenn zwei Clients angemeldet sind, werden weitere Clients abgewiesen.

3.5 Spiellogik

1. (`do-set univ wrld m`):

Die `do-set` Funktion ermöglicht das setzen eines Steines. Wenn ein Spieler einen Stein setzt, wird geprüft, ob der Zug zulässig ist und ob und wie viele Steine von dem Gegner geschlagen wurden. Geschlagene Steine werden ebenfalls von dem Spielbrett entfernt.

2. (`do-handicap univ wrld m`):

Die `do-handicap` Funktion ermöglicht das setzen der Vorgabe. Ein Spieler kann so oft einen Stein hintereinander setzen bis seine Vorgaben verbraucht sind.

3. (`calc-score board`):

Die `calc-score` Methode ermittelt am Ende des Spieles anhand des Spielfeldes das Endergebnis. Dazu werden eroberte Flächen gesucht und ausgewertet. Jede freie Position sowie jeder gegnerische Stein in einem eroberten Gebiet sind ein Punkt für den Eroberer. Diese Funktion liefert ein Pair mit den erzielten Punkten zurück. (schwarz . weiß)

4 Paketstruktur

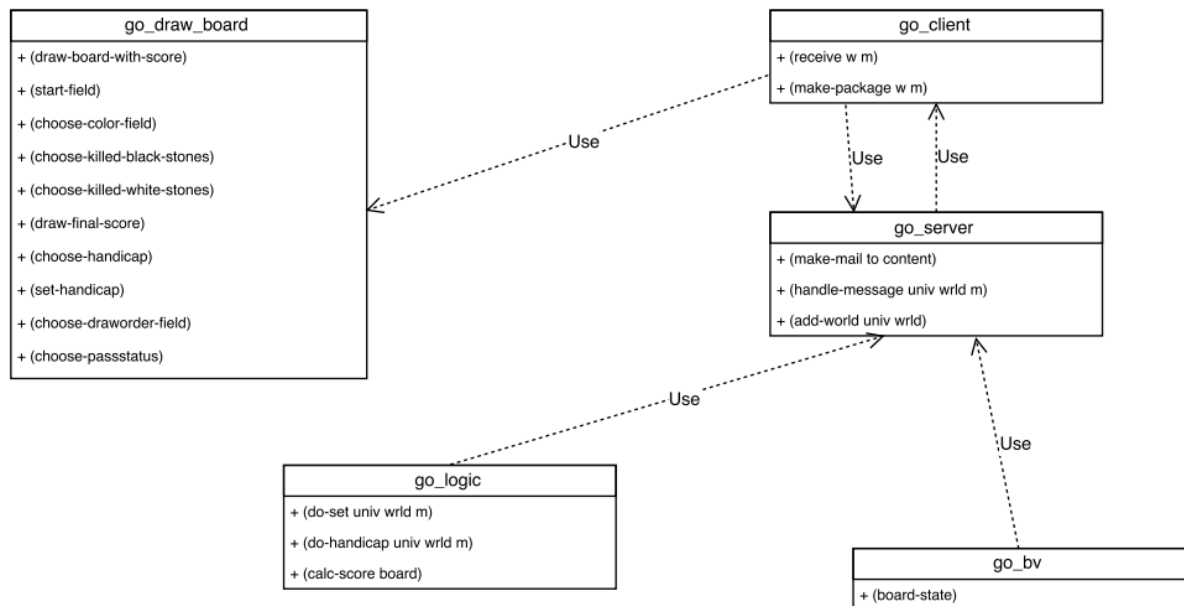


Abbildung 4.1: Paketstruktur

5 Programmabläufe

5.1 Programmablauf

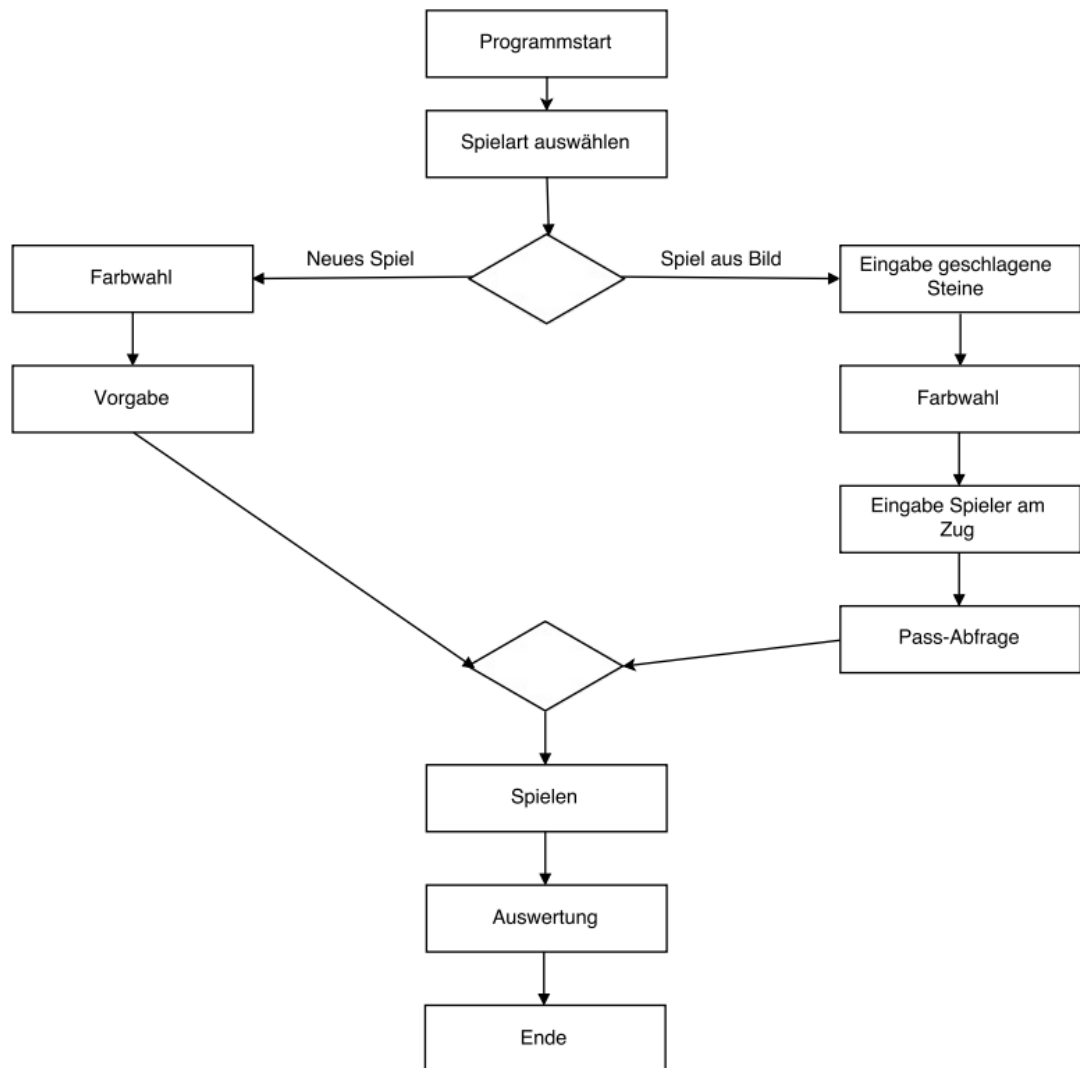


Abbildung 5.1: Ablaufdiagramm des Programmes

5.2 Stein setzen

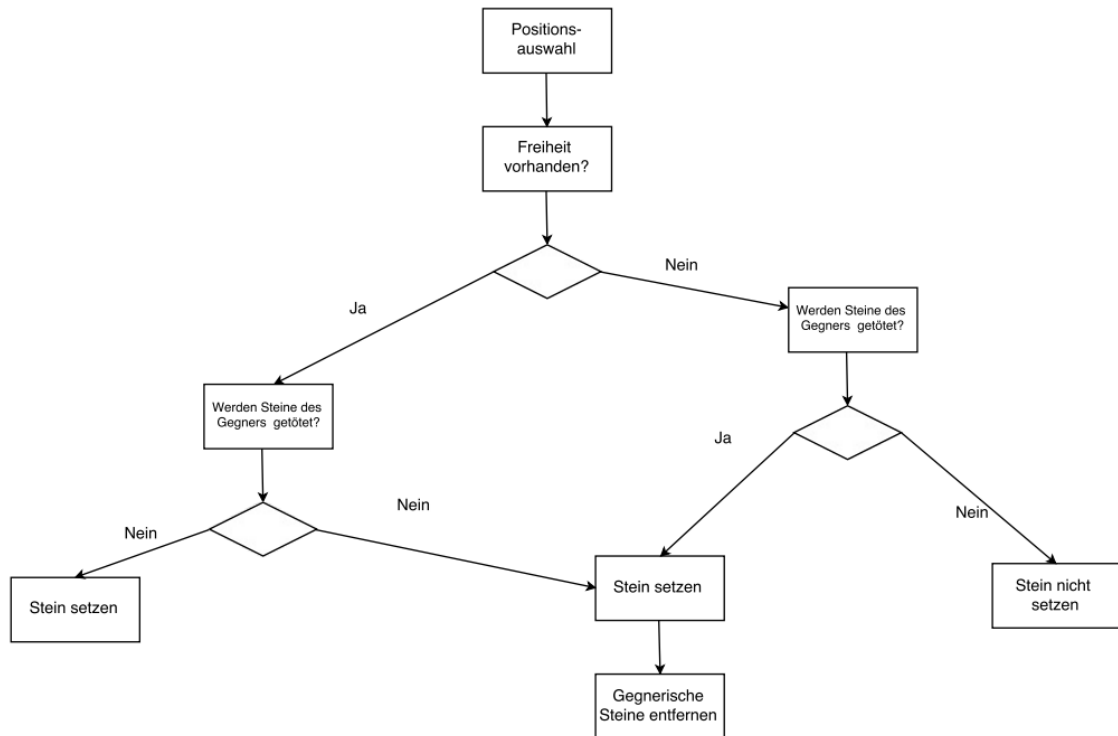


Abbildung 5.2: Ablaufdiagramm für das Setzen eines Steines

Wenn ein Spieler einen Stein setzt muss die Gültigkeit dieses Zuges geprüft werden. Wenn keine Freiheit direkt bei dem zu setzenden Stein zu finden ist, müssen alle direkten Steine der gleichen links, rechts, oben und unten von dem gesetzten Stein auf vorhandene Freiheiten überprüft werden. Diese benachbarten Steine werden als Startpunkte genommen. Von diesen werden weitere benachbarte Steine der gleichen Farbe gesucht. Diese Suche wird so lange fortgesetzt bis eine Freiheit gefunden wird. Wird eine Freiheit gefunden, kann der Stein gesetzt werden. Wird keine Freiheit gefunden, ist der Zug ungültig, weil der Spieler die eigene Struktur mit diesem Zug selbst schlagen würde. Diesen Zug nennt man dann Selbstmord und das ist nicht erlaubt.

Dieser Zug ist nur zugelassen, wenn man mit diesem Zug Steine des Gegners schlagen würde. Dazu muss der gleiche Prozess nur mit benachbarten Steinen der gegnerischen Farbe durchlaufen werden. Wird keine Freiheit gefunden, werden die Steine des Gegners geschlagen und die eigene Struktur erhält neue Freiheiten.

Die Prüfung auf geschlagene Steine des Gegners wird ebenfalls durchgeführt, wenn ein Zug sofort als gültig erkannt wird. Das heißt, dass der Zug keinen Selbstmord hervorruft, aber Steine schlägt. Das heißt, dass bei jedem regulär gesetzten Steine die benachbarten gegnerischen Strukturen auf Freiheiten überprüft werden. Findet sich bei einer Struktur wiederum keine Freiheit, werden die Steine entfernt und die Punkte werden dem setzenden Spieler hinzugefügt.

```

179 |
180 | ;;Prüft jeden gegnerischen Stein (= Start des möglichen Pfades) um den gesetzten Stein herum.
181 | ;;
182 | ;;Parameter
183 | ;;board: Das Spielbrett mit Belegung.
184 | ;;player: Der Spieler, der am Zug ist.
185 | ;;beaten_stones: Die bereits geschlagenen Steine
186 | ;;start_stones: Die umliegenden Steine um den neu gesetzten Stein, die geprüft werden müssen.
187 | ;;
188 | ;;return: Die Anzahl der geschlagene Steine und die neue Belegung des Spielfeldes
189 | (define (find-freedoms board player beaten_stones start_stones)
190 |   (if(empty? start_stones)
191 |     (cons (length beaten_stones) (kill-stones board beaten_stones));kill
192 |     (find-freedoms board player (append beaten_stones (check-freedom-path board player '() (list (car start_stones))) (cdr start_stones))
193 |     ))
194 |
195 | ;;Sucht ab des Startsteines eine Freiheit
196 | ;;
197 | ;;Parameter
198 | ;;board: Das Spielbrett mit Belegung.
199 | ;;player: Der Spieler, der am Zug ist.
200 | ;;proofed: Die bereits geprüften Steine.
201 | ;;proof: Die noch zu prüfende Steine.
202 | ;;
203 | ;;return: Wenn keine Freiheit gefunden wurde, die Liste mit den geschlagenen Steinen. Sonst die leere Liste.
204 | (define (check-freedom-path board player proofed proof)
205 |   (if (empty? proof)
206 |     proofed
207 |     (let* ((y (car (first proof)))
208 |            (x (cdr (first proof)))
209 |            (free? (check-coordinate board x y player proof proofed route-list)))
210 |       (if (equal? free? 'free)
211 |         '()
212 |         (check-freedom-path board player (cons (cons y x) proofed) (remove-list proofed (cdr free?))))
213 |       ))
214 | )

```

Abbildung 5.3: Funktionen für das Suchen von Freiheiten benachbarter Steinen

5.3 Auswertung

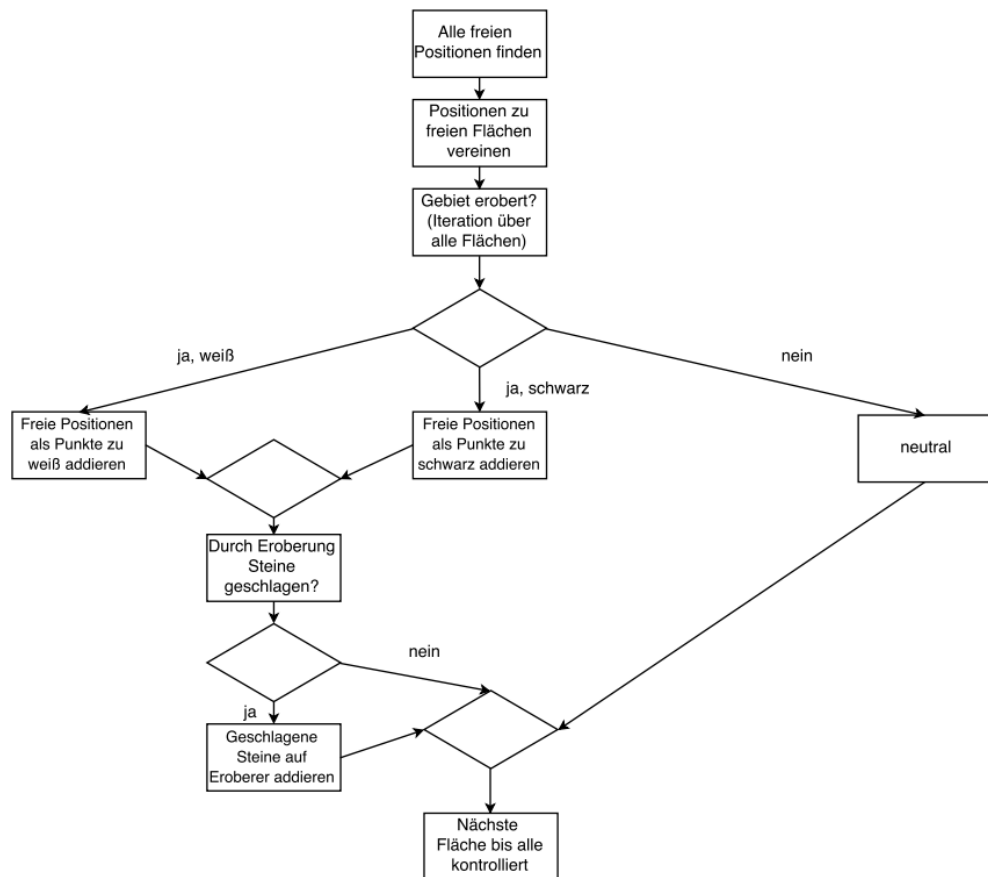


Abbildung 5.4: Ablaufdiagramm der Auswertung

Die Auswertung startet mit der Suche nach allen freien Positionen auf dem Spielfeld. Diese freien Positionen werden zu freien Flächen (=Gebiete) zusammengefasst. Anschließend wird jede freie Fläche überprüft, ob die Fläche von einem Spieler erobert wurde. Diese Funktionalität wird in dem Kapitel Gebietszuweisen (5.4) genauer erläutert. Wurde ein Gebiet von einem Spieler erobert, wird die Anzahl der freien Positionen zu den Punkten des Spielers hinzugefügt. Liegen in dem eroberten Gebiet Steine des Gegners, werden diese geschlagen und ebenfalls dem Spieler als Punkte hinzugefügt. Für das Finden der geschlagenen Steine wird das eroberte Gebiet mit den Steinen des Eroberers aufgefüllt. Für diesen Prozess werden die Funktionen zum Setzen eines Steines benutzt, um geschlagene Steine zu finden. Diese Funktionen sind in dem Kapitel Stein setzen (5.2) genauer beschrieben. Wird keinem Spieler ein Gebiet zugeschrieben, ist es neutral und kein Spieler erhält Punkte.

Diese Prüfung wird mit allen freien Flächen durchlaufen, um so den Gesamtspielstand zu bestimmen. Am Ende werden die erzielten Punkte in der Auswertung auf die geschlagenen Steine während des Spiels aufaddiert und der Sieger bestimmt.

```

391 ;;Besetzt alle leeren Flächen mit der Farbe des Spielers, der die Fläche gewonnen hat.
392 ;;Dabei wird überprüft, ob währenddessen Steine des Gegners geschlagen werden.
393 ;;Ist dies der Fall, werden diese während der Füllung des Gebietes aufaddiert und zurückgegeben.
394 ;;
395 ;;Parameter
396 ;;board: Das Spielbrett, das ausgewertet werden soll.
397 ;;empty_area: Die freie Fläche.
398 ;;last_area: Die zuletzt ausgewertete Fläche.
399 ;;player-won: Der Spieler, der die letzte Fläche gewonnen hat. (0 = neutrale Fläche)
400 ;;result: Die aufaddierten Punkte.
401 ;;
402 ;;return: Liefert die aufaddierten Punkte (geschlagene Steine) zurück,
403 ;;die während des Spielfeld Befüllens aufaddiert wurden.
404 (define (count-killed-stones-in-area board empty_area player-won result)
405   (if (empty? empty_area)
406       result
407       (let ((new-board (find-freedoms (set-stone board (caar empty_area) (cdar empty_area) player-won)
408                                       player-won
409                                       '())
410           (find-opposing-stones
411             (set-stone board (caar empty_area) (cdar empty_area) player-won)
412             (caar empty_area) (cdar empty_area)
413             player-won
414             route-list '()))))
415         (count-killed-stones-in-area (cdr new-board)
416                                     (cdr empty_area)
417                                     player-won
418                                     (+ (car new-board)
419                                       result)))
420   )
421 ))
422

```

Abbildung 5.5: Zählt die geschlagenen Steine aller eroberten Fläche

5.4 Gebietszuweisung

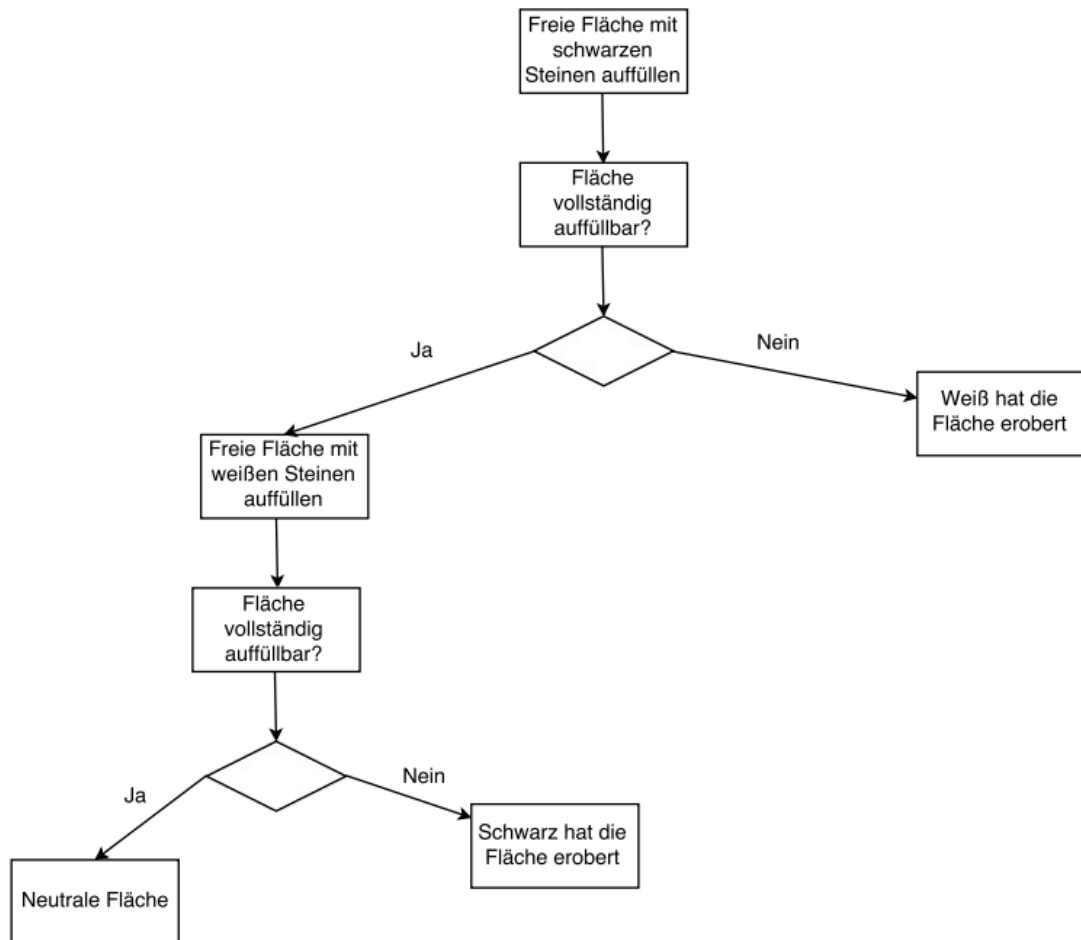


Abbildung 5.6: Ablaufdiagramm der Gebietszuweisung

Um ein Gebiet einem Spieler zuzuweisen, muss geprüft werden, ob dieser das Gebiet erobert bzw. eingeschlossen hat. Auch hier werden die Funktionen der Stein setzen Funktionalität (5.2) verwendet. Das freie Fläche wird zuerst vollständig mit schwarzen Steinen aufgefüllt. Ist die Fläche vollständig mit schwarzen Steinen auffüllbar, dann ist Schwarz ein potentieller Eroberer der Fläche, weil sie nicht von Weiß eingeschlossen ist. Wäre dies der Fall, dann wäre die Fläche nicht vollständig auffüllbar, weil Schwarz bei dem letzten Stein einen Selbstmord begehen würde. Dies ist aber in der Stein setzen Funktionalität (5.2) nicht erlaubt. Ist die Fläche also nicht vollständig auffüllbar, dann ist weiß der Eroberer dieser Fläche.

Davon ausgehend, dass Schwarz ein potentieller Eroberer ist, muss anschließend noch geprüft werden, ob Weiß die Fläche ebenfalls auffüllen darf. Darf Weiß dies nicht, gehört die Fläche Schwarz. Darf Weiß die Fläche ebenfalls komplett auffüllen, ist die Fläche neutral, weil sie somit von keiner Farbe erobert ist.

```

343
344 ;;Iteriert über alle leeren Flächen und addiert die erzielten Punkte.
345 ;;
346 ;;Parameter
347 ;;board: Das Spielbrett, das ausgewertet werden soll.
348 ;;empty_areas: Die freien Flächen auf dem Spielbrett.
349 ;;last_area: Die zuletzt ausgewertete Fläche.
350 ;;player-list: Liste mit den beiden Spielern (nur für iteration).
351 ;;player-won: Der Spieler, der die letzte Fläche gewonnen hat. (0 = neutrale Fläche)
352 ;;points: Die Punkte als Pair (Punkte für Schwarz . Punkte für Weiß) gespeichert.
353 ;;
354 ;;return: Das Ergebnis der Auswertung als Pair (Punkte für Schwarz . Punkte für Weiß)
355 (define (check-all-empty-areas board empty_areas last_area player-list player-won points)
356   (let ((new_points
357         (cond
358           [(eqv? player-won -1) (cons (calc-points (car points) last_area board player-won)
359                                       (cdr points))]
360           [(eqv? player-won 1)  (cons (car points)
361                                       (calc-points (cdr points) last_area board player-won))]
362           [else points])))
363     (cond
364       [(empty? empty_areas) new_points]
365       [else (check-all-empty-areas board
366                                     (cdr empty_areas)
367                                     (car empty_areas)
368                                     player-list
369                                     (check-empty-area-for-each-player board (car empty_areas) player-list #t)
370                                     new_points)]
371     )))
372

```

Abbildung 5.7: Prüft alle leeren Flächen auf eine Eroberung und addiert die freien eroberten Positionen zu den Punkten hinzu

6 Beispiele

6.1 Erlaubter Selbstmord

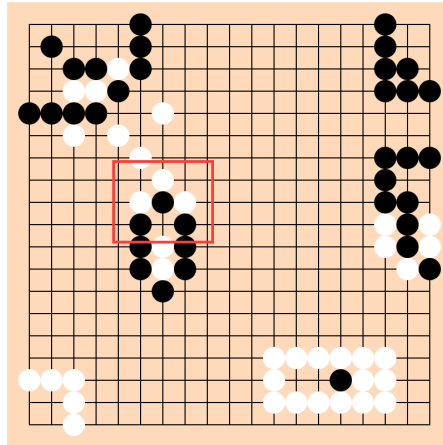


Abbildung 6.1: Spielbrett vor erlaubten Selbstmord

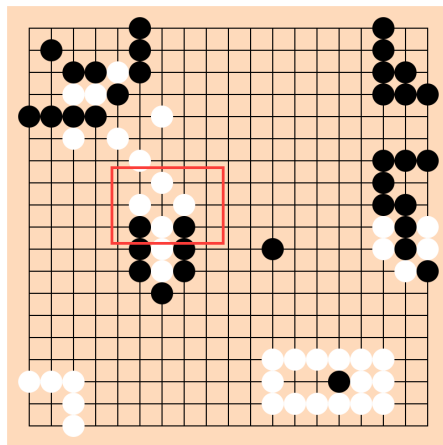


Abbildung 6.2: Spielbrett nach dem erlaubten Selbstmord

6.2 Auswertung

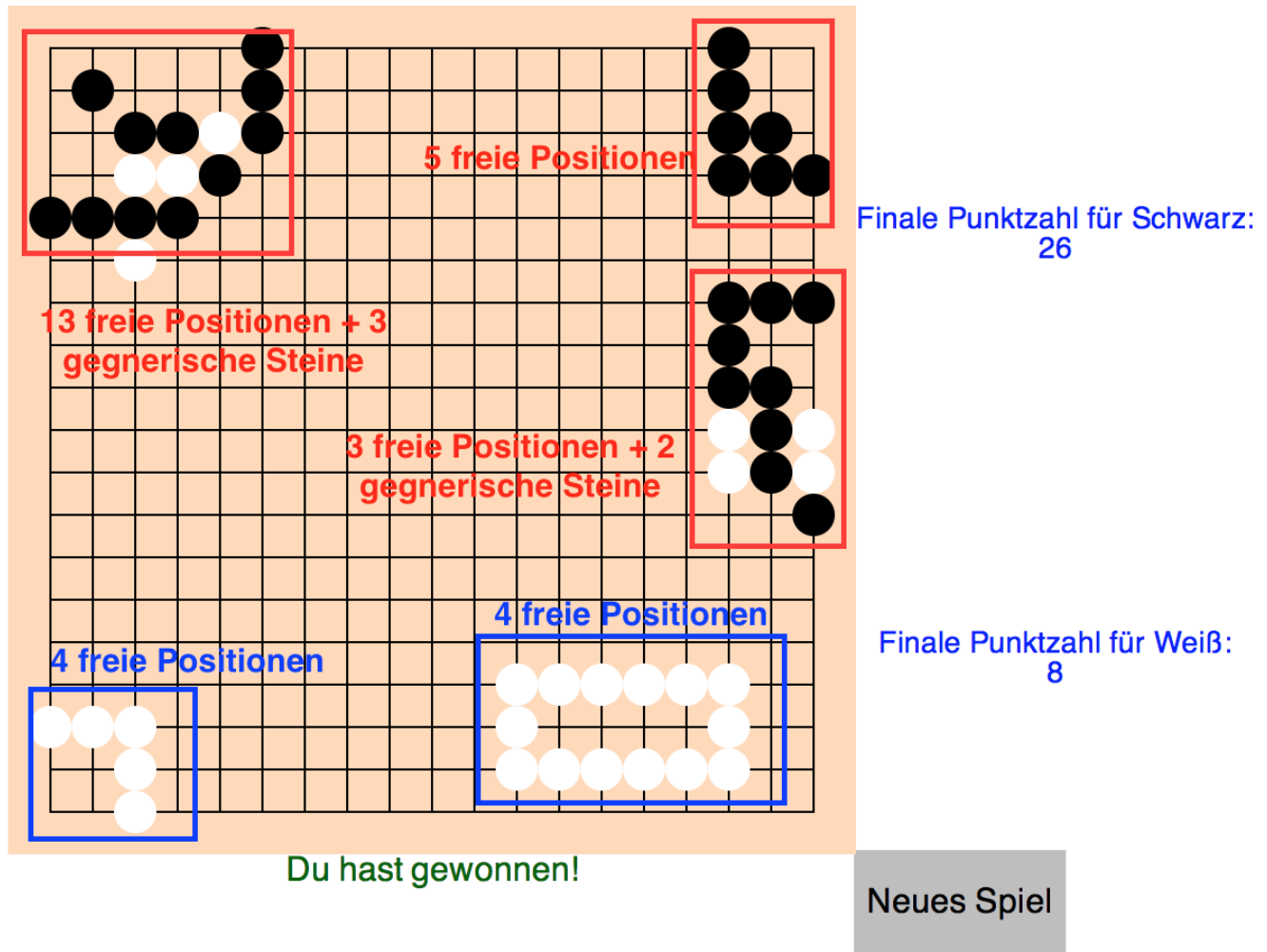


Abbildung 6.3: Beispielhafte Auswertung

7 ToDo's

7.1 Dateiverarbeitung (go_datei.rkt)

Dieses Modul lädt aus einer Spielstand-Datei den aktuellen Spielstand und gibt diesen als World (1.2) zurück. Ebenfalls kann dieses Modul einen Spielstand in die Datei schreiben. Die Datei enthält folgende Informationen:

1. Belegung des Spielfeldes (1.1)
2. Anzahl geschlagener Steine (schwarz . weiß)
3. Der Spieler, der am Zug ist
4. Passstatus des vorherigen Spielers

7.2 Bug in Auswertung

Aufgrund der hohen Komplexität der Auswertung war keine Zeit mehr einen Sonderfall in der Auswertung zu behandeln. Dieser tritt auf wenn eine eroberte Fläche durch gegnerische Steine unterbrochen wird (Abb. 6.1). Die Funktion, die die leeren Flächen sucht, erkennt dieses Gebiet als zwei leere Flächen, die von keiner Partei erobert wurden. Normalerweise muss bei der Auswertung erkannt werden, dass es sich nur um eine eroberte Fläche handelt und die gegnerischen Steine müssen geschlagen werden. Lösungsansatz:

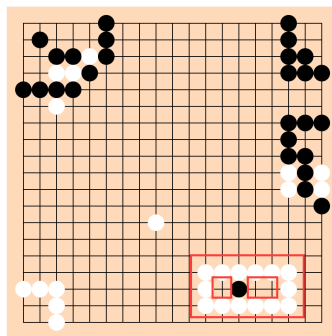


Abbildung 7.1: Spielbrett nach dem Schlagen des Steines

Es müssen drei Bedingungen geprüft werden, damit zwei getrennte Flächen als gemeinsame erkannt werden müssen:

1. Beide Flächen dürfen einzeln noch nicht erobert sein
2. Sie müssen über gegnerische Steine miteinander verbunden sein
3. Gemeinsam mit den gegnerischen Steinen muss die Fläche als erobert gelten

Treten alle drei Fälle ein, dürfen die leeren Flächen als gemeinsame betrachtet werden. Dann bekommt der Eroberer sowohl die Punkte für die leeren Positionen als auch die Punkte für die geschlagenen Steine.

Diese Änderung muss in der Spiellogik implementiert werden, bevor die eigentliche Auswertung startet. Dann müssen alle gefundenen freien Flächen geprüft werden, ob sie zusammen gehören und anschließend in der Auswertung auch als gemeinsame Fläche betrachtet werden. Wir erachten den Aufwand als erheblich.